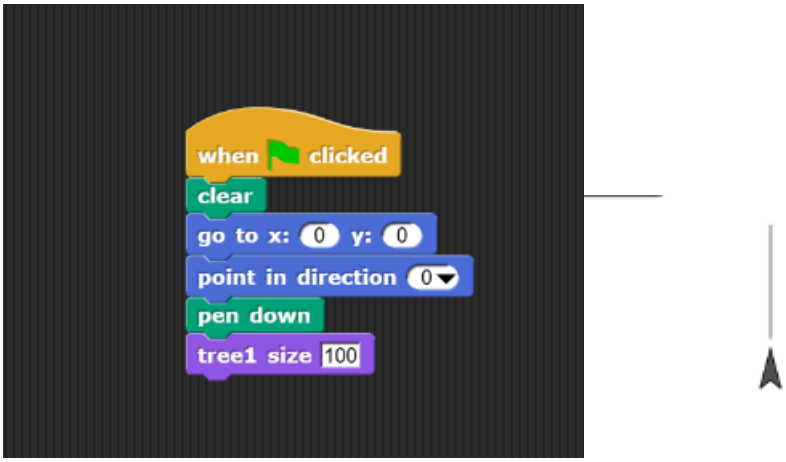
1. Make a tree1 block:

- **Purpose:** This block will draw a single vertical line (the simplest "trunk") and then return the sprite to its starting position and orientation. This "return to start" behavior is *critical* for recursion later.
- **Input:** It needs one input, which the instructions call **size**. This will determine the length of the line.

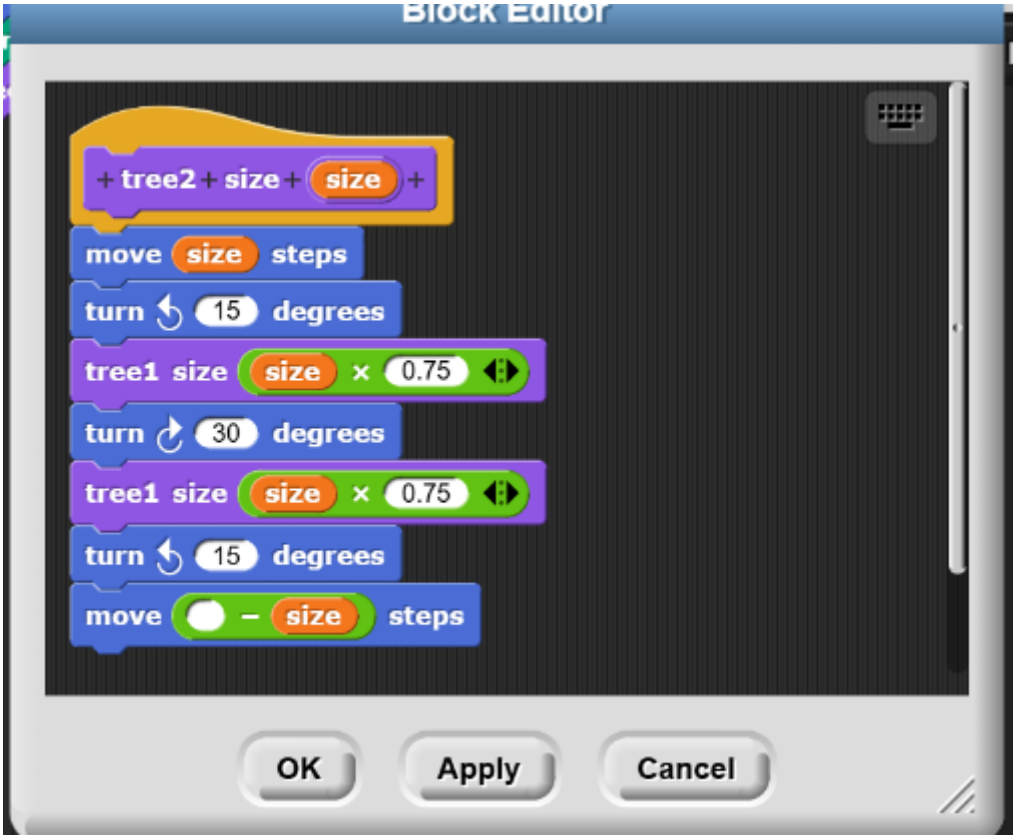


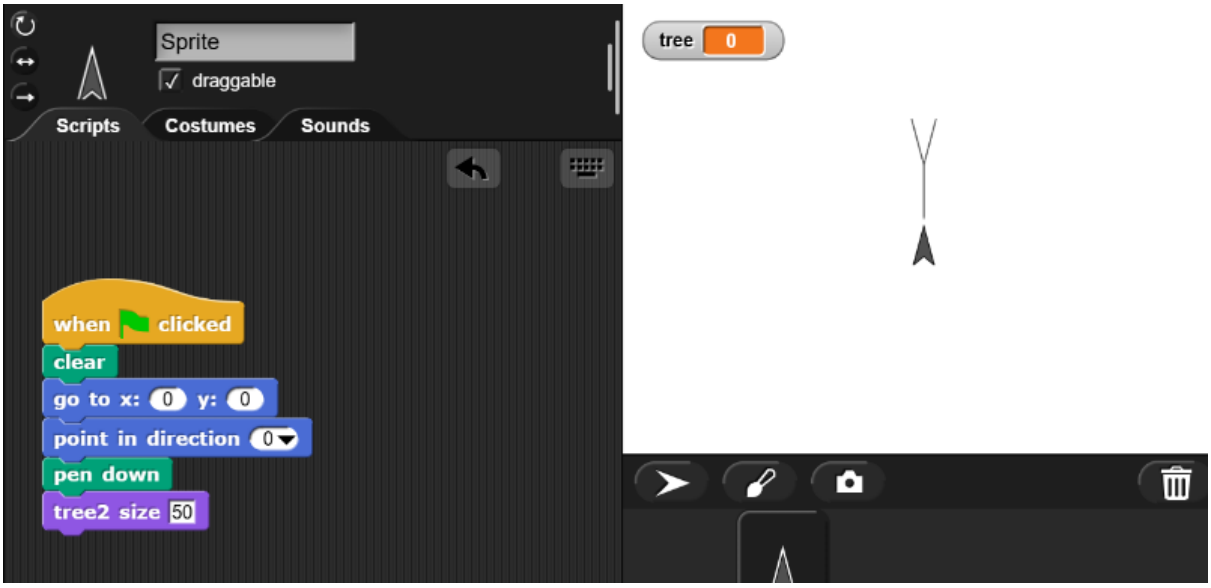
Make a tree2a block that draws two levels.

- **Purpose:** This block will draw a main trunk, then turn and draw two smaller "branches" at angles, returning the sprite to its start position and orientation.
- **Input:** It also needs one input, **size**.

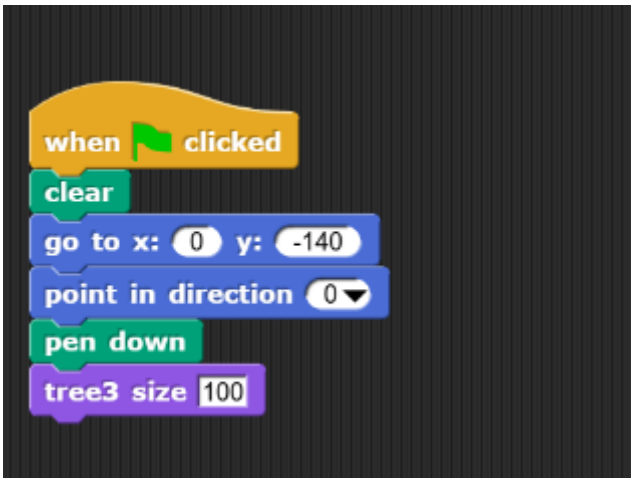


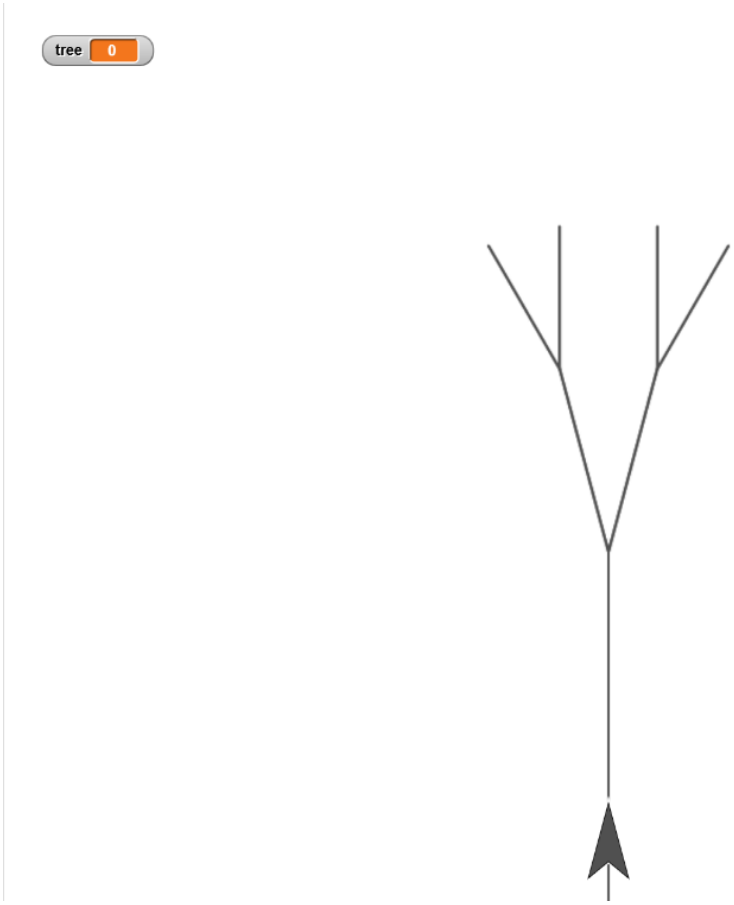
That tree2a block is pretty long and repetitive. But we can simplify it... So we can use tree1 to shorten tree2. Compare the code below to convince yourself that the new tree2 will work the same as the original tree2a.


Block Editor
+ tree2 + size + size +
move size steps
turn 15 degrees
tree1 size size × 0.75
turn 30 degrees
tree1 size size × 0.75
turn 15 degrees
move – size steps
OK
Apply
Cancel


Sprite
draggable
Scripts
Costumes
Sounds
when clicked
clear
go to x: 0 y: 0
point in direction 0
pen down
tree2 size 50
tree 0

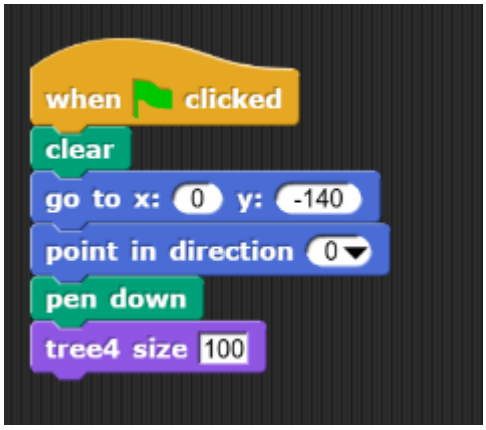
Make a tree3 block that uses the tree2 block, on the same pattern. Don't forget about the "duplicate" feature in Snap!, which will save you a bunch of time making tree3. Once you're done, try out tree3 and you should see output that looks something like the following: (The image for **tree3** output is not shown here, but it would be a tree with three levels of branching).



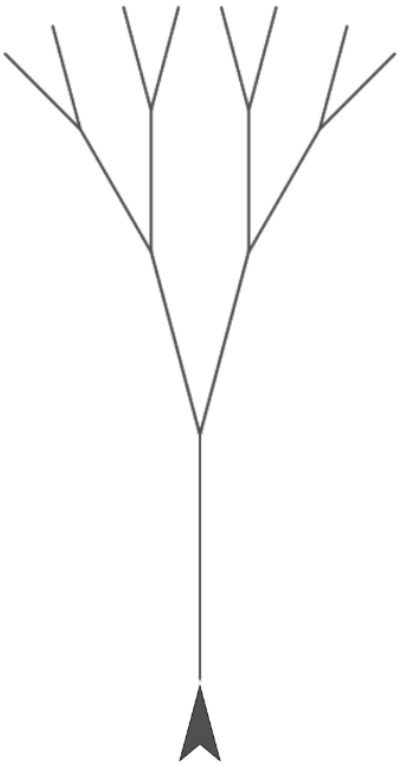


make a tree4 block that uses the tree3 block and try it out.

- **Purpose:** This is to really hammer home the pattern before we generalize it.

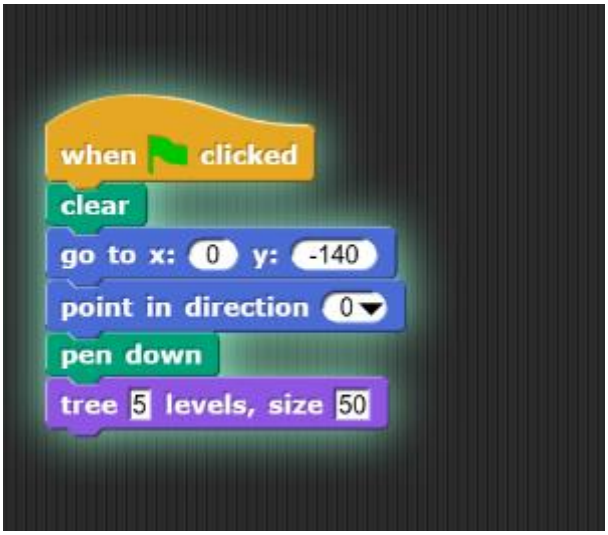




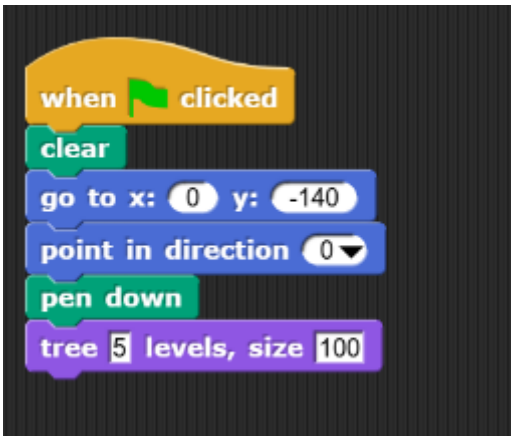


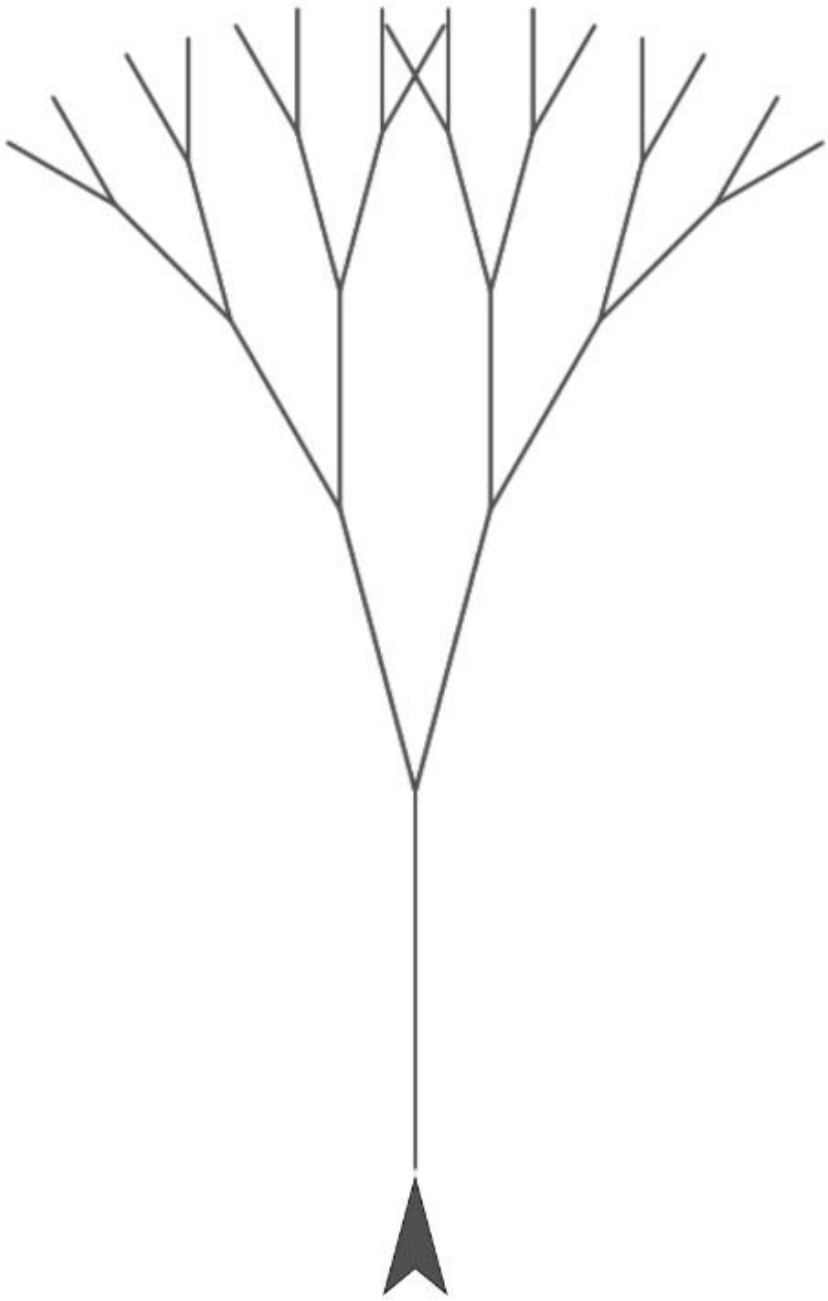
it makes sense to wonder if we can replace them all with a single tree block. Here's the big idea: We can write a tree block that uses itself in its own script provided that it knows how many levels it's expected to draw! So, in addition to the size input, it'll have a levels input.

- **Purpose:** Create a single, general, recursive **tree** block.



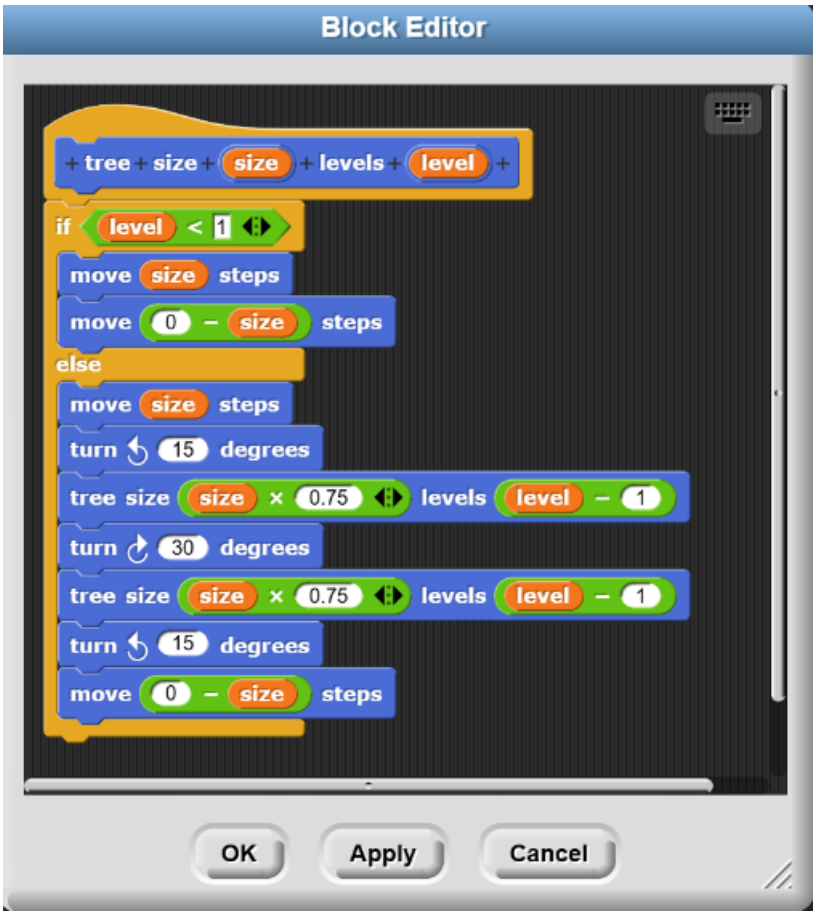


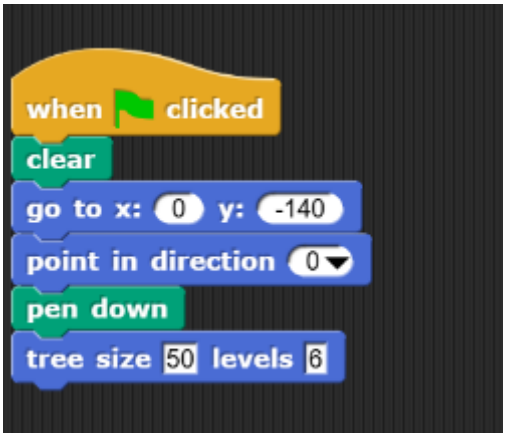

when clicked
clear
go to x: 0 y: -140
point in direction 0
pen down
tree 5 levels, size 100

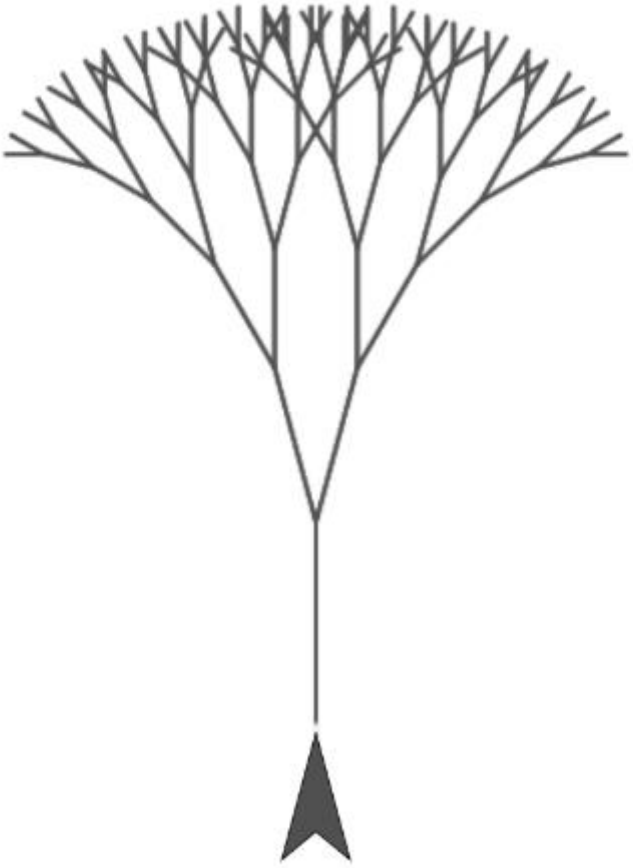


This script defines a **tree** block that:

- **If level is 0 (base case):** Draws a simple line and returns.
- **If level is 1 or more (recursive case):**
    1. Draws its own segment.
    2. Turns and calls itself to draw a smaller left sub-tree with **level-1**.
    3. Turns and calls itself to draw a smaller right sub-tree with **level-1**.
    4. Returns to its starting position and orientation for that segment.







1. What would happen if we ran our tree block with a level input of less than 1? Think about it before you try it out.

- Let's look at the structure of that **tree** block:

define tree size (size) levels (level)

```
if < (level) < (1) > then       // Condition: is level 0, -1, -2, etc.?
    // Base Case Action
    move (size) steps
    move ((0) - (size)) steps
 else
    // Recursive Case Action (for level 1, 2, 3, etc.)
    // ... (draws trunk, turns, calls tree for sub-branches, turns back, moves back) ...
 end
end
```

- **If level is 0:**
    - The condition **(0) < (1)** is TRUE.
    - The script will execute the "Base Case Action":
        - **move (size) steps**
        - **move ((0) - (size)) steps**
    - So, it will draw a single line segment of the given **size** and then immediately move back along that line, ending where it started for that segment. No further recursive calls are made.
- **If level is -1 (or any other negative number):**
    - The condition **(-1) < (1)** (or any negative number < 1) is TRUE.
    - The script will execute the same "Base Case Action" as when **level** is 0.
    - It will draw a single line segment of the given **size** and move back.
- **Conclusion:** If the **tree** block (as defined in the previous image with the **if < (level) < (1) >** condition) is run with a **level** input of less than 1 (e.g., 0, -1, -5), it will simply draw a straight line of the current **size** and then retrace that line, effectively drawing a single segment and returning the sprite to its start. It will **not** enter the **else** part and therefore will not make any recursive calls. The "tree" will just be a single stem.

2. Sometimes you can make a recursive procedure more elegant by having a smaller base case. In our tree program, what would a "zero level" tree mean? Try rewriting tree so that the base case is level=0.

- **What would a "zero level" tree mean?**
    - Logically, a "zero level" tree should mean "no tree at all" or "do nothing." It's the point at which you stop adding any more structure.
    - In the *current* script (from the previous image), a **level = 0** call *does* draw a small segment (the base case action).
    - If a "zero level" tree truly means "do nothing," then the action for the base case **level=0** should be empty.
- **Try rewriting tree so that the base case is level=0 (and implies "do nothing" at level 0):**

The most straightforward way to achieve this is to change the main condition to only execute the drawing and recursion if **level** is greater than 0.

Elegant tree block (base case level=0 means do nothing):

```
define tree size (size) levels (level)
  if < (level) > (0) > then  // Only do something if level is 1 or more
    // These are lines 6-12 from the previous script image
    move (size) steps
    turn ↺ (15) degrees
    tree size: ((size) * (0.75)) levels: ((level) - (1)) // Recursive call
    turn ↻ (30) degrees
    tree size: ((size) * (0.75)) levels: ((level) - (1)) // Recursive call
    turn ↺ (15) degrees
    move ((0) - (size)) steps
  end
  // If level is 0 (or less), the 'if' condition is false, and nothing happens.
  // This implicitly makes "level = 0" the base case where the recursion stops AND nothing is drawn.
  end
```

How this works:

- If you call **tree size: S levels: 0**, the **if < (0) > (0) >** condition is false. The block does nothing. This is our "zero level tree."
- If you call **tree size: S levels: 1**, the **if < (1) > (0) >** condition is true.
    - It draws its trunk (**move (size) steps**).
    - It turns and calls **tree size: (S*0.75) levels: 0** (for the left branch). This call does nothing.

- It turns and calls **tree size: (S*0.75) levels: 0** (for the right branch). This call also does nothing.
- It turns back and moves back.
- So, a **level=1** call effectively draws just a single trunk (or the outermost branches/leaves if you consider it from the top down).

This "elegant" version is actually very similar to the first recursive **tree** block suggested in instruction #7 from a few steps ago, before the explicit **if/else** with a drawing base case was introduced. The key difference in interpretation is what happens when **level** finally reaches the point where recursion stops. In this "elegant" version, it stops *and does nothing further*.